Q-Learning CIA Test 1 — FrozenLake Debugging

Marks:15

Instructions

The following program implements a **Q-learning delivery robot using a custom 4 × 4 FrozenLake environment**.

- Start state = 3
- Goal state = 12
- Correct the errors without completely rewriting the program.
- Execute the corrected program and verify the learned route and final reward.

Environment

```
F  F  F  S
F  F  H  F
F  F  F  F
G  F  F  F
```

State numbering is row-wise from 0 to 15.

In [8]:
```python
# Q-Learning using a custom FrozenLake environment

import numpy as np
import random
import gymnasium as gym

custom_map = [
    "FFFS",
    "FFHF",
    "FFFF",
    "GFFF"
]

env = gym.make(
    "FrozenLake-v1",
    desc=custom_map,
    is_slippery=True
)

n_states = env.observation_space.n
n_actions = env.action_space.n

Q_table = np.ones((n_states, n_actions))

learning_rate = 0.1
discount_factor = 0.99
epsilon = 1.0
episodes = 10000
```

```python
# 0 = Left, 1 = Down, 2 = Right, 3 = Up

def choose_action(state):
    if random.random() < epsilon:
        return random.randrange(n_actions)       # exploration
    else:
        return np.argmax(Q_table[state])         # exploitation


for episode in range(episodes):

    state, info = env.reset()
    epsilon = max(0.01, epsilon * 0.995)


    for step in range(100):

        action = choose_action(state)

        next_state, reward, terminated, truncated, info = env.step(action)

        done = terminated or truncated

        best_next_q = np.max(Q_table[next_state])


        target = reward + discount_factor * best_next_q


        if terminated:
            target = reward

        Q_table[state, action] = (
            Q_table[state, action]
            + learning_rate *
            (target - Q_table[state, action])
        )

        state = next_state

        if done:
            break

print("Learned Q-table:")
print(np.round(Q_table, 3))

action_symbols = {
    0: "←",
    1: "↓",
    2: "→",
    3: "↑"
}

grid = env.unwrapped.desc

print("\nLearned Delivery Robot Policy")
print("-----------------------------")
```

```
for row in range(4):

    line = ""

    for col in range(4):

        state = row * 4 + col
        cell = grid[row][col].decode()

        if cell == "S":
            symbol = "S"
        elif cell == "G":
            symbol = "G"
        elif cell == "H":
            symbol = "X"
        else:
            best_action = np.argmax(Q_table[state])
            symbol = action_symbols[best_action]

        line += f"[ {symbol} ] "

    print(line)

state, info = env.reset()

start_state = state
path = [state]
total_reward = 0

for step in range(30):

    action = np.argmax(Q_table[state])

    next_state, reward, terminated, truncated, info = env.step(action)

    total_reward += reward
    path.append(next_state)

    state = next_state

    if terminated or truncated:
        break

print("\nStart state:", start_state)
print("Path followed:", path)
print("Total reward:", total_reward)

goal_state = 12

if state == goal_state:
    print("Delivery successful!")
else:
    print("Delivery unsuccessful.")

env.close()
```

```
Learned Q-table:
[[0.887 0.866 0.861 0.865]
 [0.876 0.849 0.855 0.847]
 [0.514 0.621 0.732 0.84 ]
 [0.808 0.821 0.805 0.806]
 [0.884 0.912 0.879 0.878]
 [0.894 0.588 0.529 0.648]
 [1.    1.    1.    1.   ]
 [0.609 0.631 0.822 0.577]
 [0.92  0.945 0.922 0.916]
 [0.928 0.897 0.901 0.893]
 [0.351 0.874 0.656 0.71 ]
 [0.85  0.858 0.85  0.853]
 [1.    1.    1.    1.   ]
 [0.943 0.925 0.915 0.92 ]
 [0.874 0.871 0.873 0.873]
 [0.854 0.86  0.853 0.854]]

Learned Delivery Robot Policy
------------------------------
[ ← ] [ ← ] [ ↑ ] [ S ]
[ ↓ ] [ ← ] [ X ] [ → ]
[ ↓ ] [ ← ] [ ↓ ] [ ↓ ]
[ G ] [ ← ] [ ← ] [ ↓ ]

Start state: 3
Path followed: [3, 2, 2, 2, 1, 1, 5, 9, 13, 13, 9, 13, 12]
Total reward: 1
Delivery successful!
```